Campus Café — Serverless Order Tracking on Azure

Project proposal · George Brown College · Work Integrated Project · October 2026

Serverless | Event-driven (queue trigger) | 3 functions · 18 resources | est. under $1 / month

Overview

A campus café needs to take customer orders and keep customers informed. This project builds a small order-tracking system on Azure using serverless services. A customer places an order. Café staff move it through four statuses: **Accepted** → **Preparing** → **Ready for pickup** → **Collected**. The customer receives an email at each of the first three.

Goals and scope

- A working, deployed system on Azure: order API, staff status API, email notifications.
- A serverless, event-driven design (a queue message triggers the email function) that shows reliability, security and monitoring in practice: retries, poison queue, managed identity, alerts.
- Near-zero cost on an Azure for Students subscription, following Azure Well-Architected principles.
- **Out of scope:** payments, customer accounts, mobile app, order-status web page.

How it works (numbers match the diagram on page 2)

1. The customer sends an order: `POST /api/orders` over HTTPS.
2. `SubmitOrder` validates it, saves it as `Accepted` in the Table, puts a message on Queue `notify`, and returns the order id.
3. Staff send a status update: `PUT /status` with a function key.
4. `UpdateOrderStatus` allows only the next valid status, saves it, and queues a message (none for `Collected`).
5. The queue message triggers `SendNotification`. A failed message retries, then moves to the poison queue and raises an alert.
6. `SendNotification` builds the email and sends it through Azure Communication Services, then marks that status as notified.
7. The customer receives the email (three per order).

Plan

Phase	Work	Output
1 Foundation	Resource group, storage, monitoring, identity	Wired environment
2 Core flow	Order + status functions, queue, table	Statuses advance
3 Email	Notification function, Communication Services	3 customer emails
4 Monitoring	Diagnostic setting, alerts, budget	Failure + poison alerts
5 Test and demo	Demo script, README	Live demo and report

Architecture at a glance

One Azure subscription holds one resource group, `rg-campus-cafe`, with 18 resources in four groups:

- **Compute:** Flex Consumption plan + one Function App (3 functions).
- **Data:** one storage account: Table `orders`, Queue `notify` (+ poison queue), Blob containers.
- **Email:** Azure Communication Services, Email service, Azure-managed domain.
- **Platform:** managed identity, Application Insights, Log Analytics, diagnostic setting, 2 alert rules, action group, budget.

Design principles

- **Serverless, event-driven:** pay per execution; the queue decouples HTTP from email.
- **Reliable:** retries, poison queue, alert; no duplicate emails on retry.
- **Secure:** HTTPS, function key for staff, managed identity + RBAC, no storage keys.
- **Observable:** Application Insights + Log Analytics; alerts email the team.
- **Cost-aware:** free grants, LRS, max 5 instances, $5 budget alert.

Estimated cost per month

Item	Estimate
Azure Functions (Flex Consumption)	Within free grant (250,000 executions, 100,000 GB-s)
Storage account (LRS)	Cents
Communication Services Email	$0.00025 per email
Application Insights + Log Analytics	Within 5 GB free
Alerts, identity, budget	Free tier; optional log alert about $0.50+
Total	**Under $1, covered by Azure for Students ($100 credit)**

Estimates; confirm in the Azure Pricing Calculator.

Risks and limits

- **Email cap:** managed domain allows 5 emails/min, 10/hour (demo under ~3 orders/hour). ACS Email retires 30 Sep 2028 and new onboarding is limited from 23 Oct 2026, so email sits behind one function and can be swapped.
- **Public endpoints:** the order API is anonymous (inputs validated); a function key is a shared secret.
- **Cold start:** the first request after idle can take a few seconds.
- **To confirm:** Flex region availability; budgets on the student subscription.

Possible later phases: Event Grid fan-out (SMS, dashboard), Key Vault, Entra ID sign-in for staff, VNet and private endpoints, Bicep and GitHub Actions.

Architecture

Infrastructure behind the order-tracking flow: subscription, resource group and all 18 resources

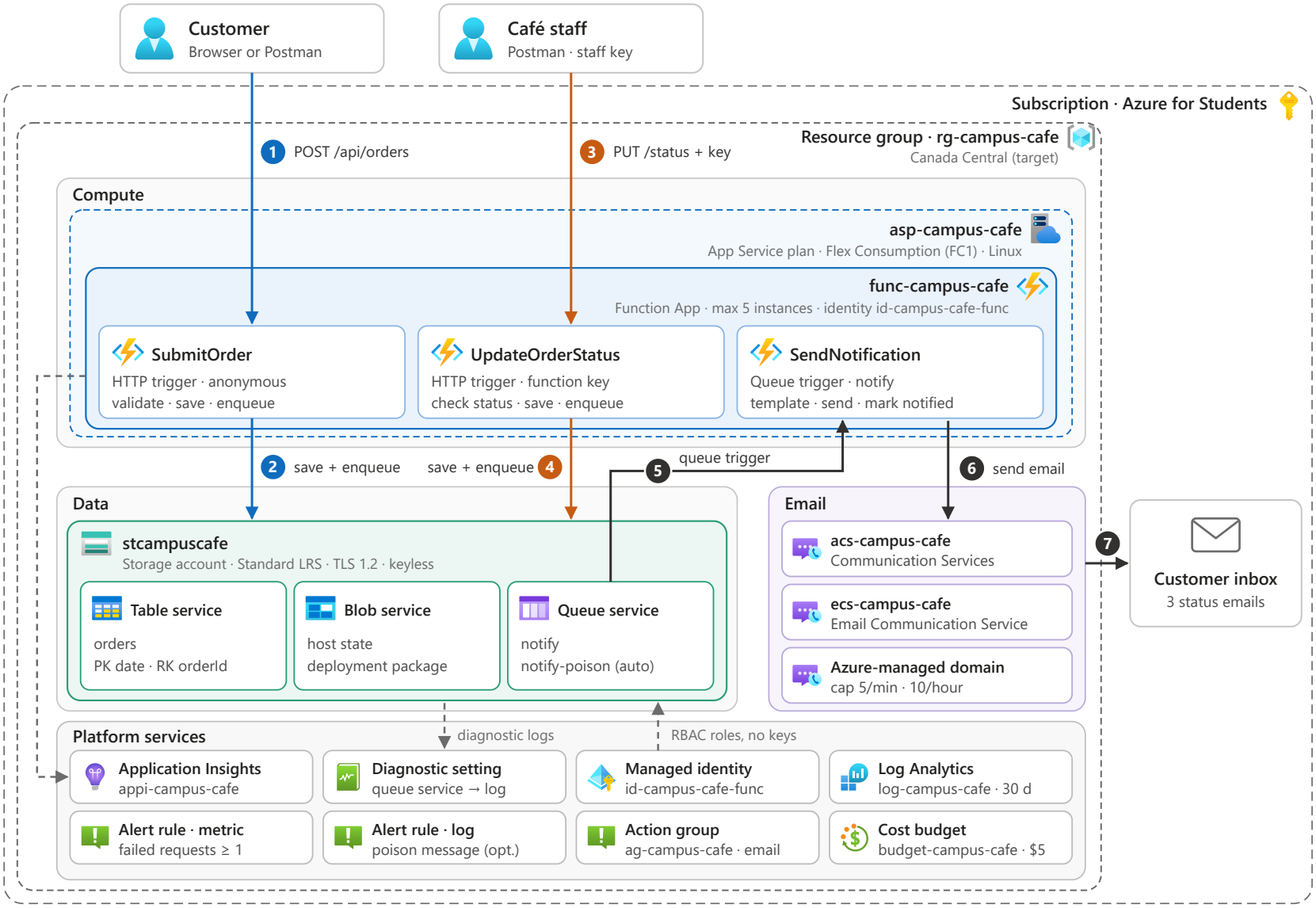


Figure 1. Numbers 1–7 follow the flow on page 1: customer order = 1–2 (blue), staff status update = 3–4 (orange), notification = 5–7 (shared). Dashed lines = identity (RBAC), diagnostic logs and telemetry.

- **Icons:** Microsoft Azure Public Service Icons V24, file `Azure_Public_Service_Icons_V24.zip` (Azure architecture icons, © Microsoft). Used unmodified, with the product name beside each icon, for architecture documentation, as the pack's Terms of Use permit.
- **Microsoft documentation:** Flex Consumption · Functions connections and roles · Queue trigger · ACS service limits · ACS retirement guide · Azure for Students